

Phase 0 — same four beats on CartPole / Flappy

Dr Nguyen's board: **predict** → **score** → **differentiate** → **update**. Games only change the environment. The training boards stay in that shape. CartPole: $s \in \mathbb{R}^4$, $A = \{L, R\}$, $r = +1/\text{step}$. Flappy: $s \in \mathbb{R}^{12}$, $A = \{\text{noop}, \text{flap}\}$.

His board

```
net = ResNet18()
opt = Adam(...)

# (x,y) fixed
preds = net(x)
loss = MSE(preds,y)/N

opt.zero_grad()
loss.backward()
opt.step()
```

REINFORCE (scratch)

```
policy = PolicyNet()
opt = Adam(policy)

# data by acting
traj = ROLLOUT(env, policy)
G = discounted_returns(traj)
L = -sum(log_pi * G)

opt.zero_grad()
L.backward()
opt.step()
```

PPO (games = trading board)

```
policy = PolicyNet()
critic = ValueNet()
opt_pi, opt_V = Adam(), Adam()

B = ROLLOUT(env, policy)
A = advantages(B, critic)
L_pi = - PPO_CLIP(policy,B,A)
L_V = MSE(V(s), returns)

opt_pi.zero_grad(); L_pi.backward(); opt_pi.step()
opt_V.zero_grad(); L_V.backward(); opt_V.step()
```

Tabular Q (binned CartPole only)

```
Q = zeros(bins, actions)

a = eps_greedy(Q[bin(s)]) # predict
s2, r, done = env.step(a)
y = r + gamma * max(Q[bin(s2)]) # score
Q[bin(s),a] += alpha*(y - Q[bin(s),a]) # update
```

DQN

```
Q, Q_bar = QNet(), copy(Q)
buf = ReplayBuffer()

a = eps_greedy(Q(s)) # predict
buf.add(s,a,r,s2,done)
y = r + gamma*max(Q_bar(s2)) # score
L = MSE(Q(s,a), y)
opt.zero_grad(); L.backward(); opt.step()
```

Key translation (games Phase 0)

- Labels y become rewards r from `env.step`.
- Fixed dataset becomes a rollout that depends on π_θ .
- Same PPO board as trading: only `ROLLOUT(env)` vs `ROLLOUT(prices)`.
- Tabular Q needs binning — reason we do not use it for stocks.